Dependencies and developer tools

Software Management & Development



IE University · MFT & MCSDI

Editor

Editor and IDE

Text editor
nano, TextEdit

→
+ colours, completion, extensions

Code editor
VS Code, Zed, Vim

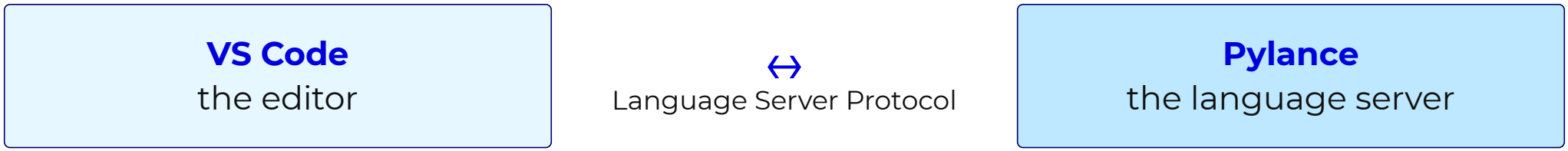
→
+ debugger, tests, Git built in

IDE
PyCharm, Visual Studio

VS Code with the Python extension works like an IDE for Python.

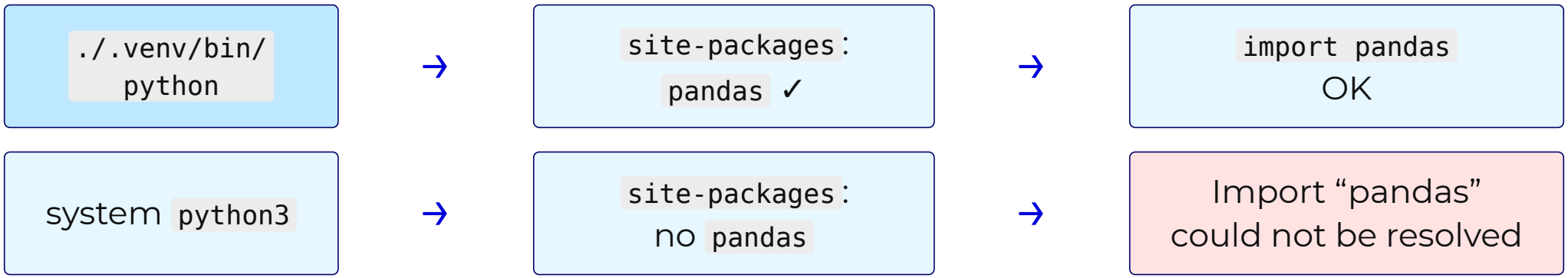
> The project is still plain text files: any editor can open it.

Language server



- Completion: `Decim` → `Decimal`
- Documentation when you hover
- Go to definition, rename everywhere
- Errors **before** you run: `"transaction" is not defined`

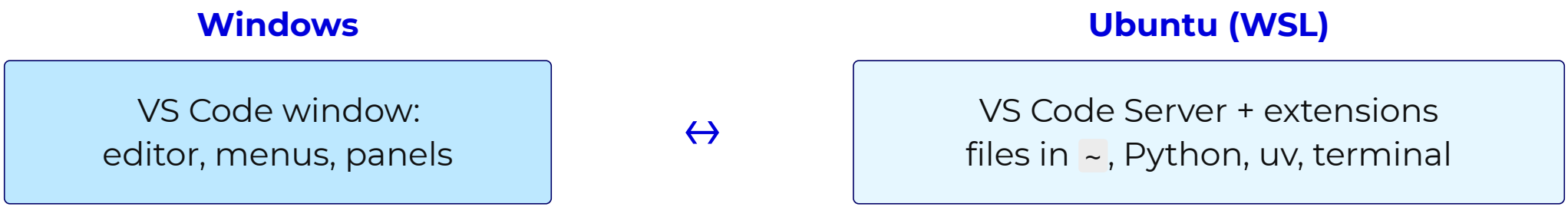
The interpreter the editor uses



Pylance looks for imported libraries in the selected interpreter.

> Command Palette → **Python: Select Interpreter** → `./.venv/bin/python`

VS Code with WSL (advanced)



- Open from the Ubuntu terminal: `code .`
- Bottom-left corner: **WSL: Ubuntu**

> In the Linux VM, VS Code runs inside Linux and needs none of this. The same idea works for a server over SSH, a container or a codespace.

Dependencies

Version constraints

`pyproject.toml` says which versions we **accept**. People write it:

```
dependencies = ["rich>=15.0.0"]
```

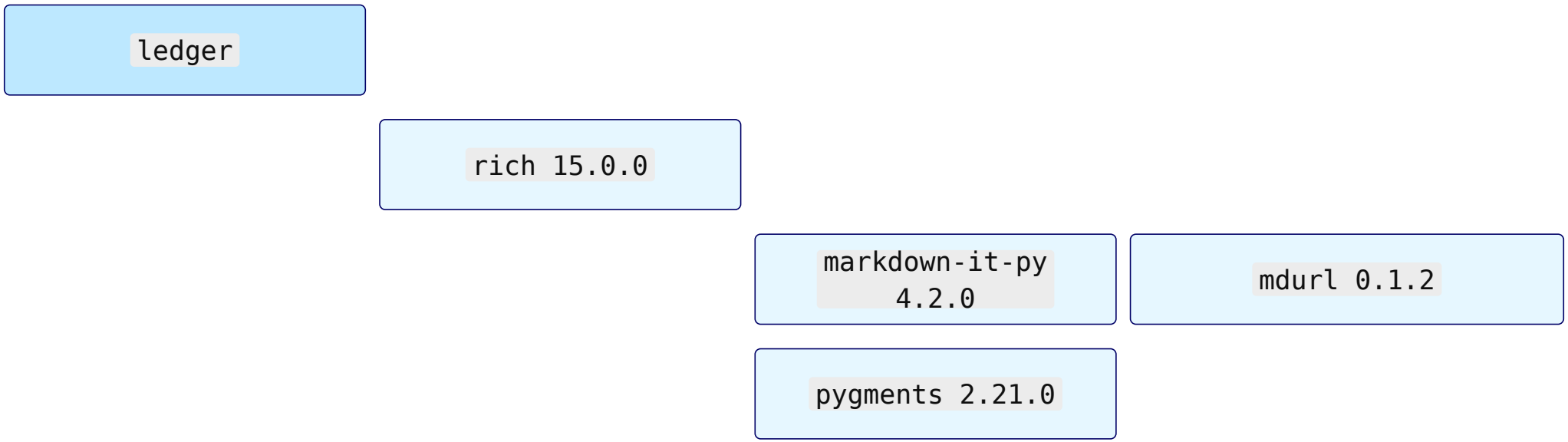
15 . **0** . **0**

MAJOR: may break | MINOR: adds | PATCH: fixes

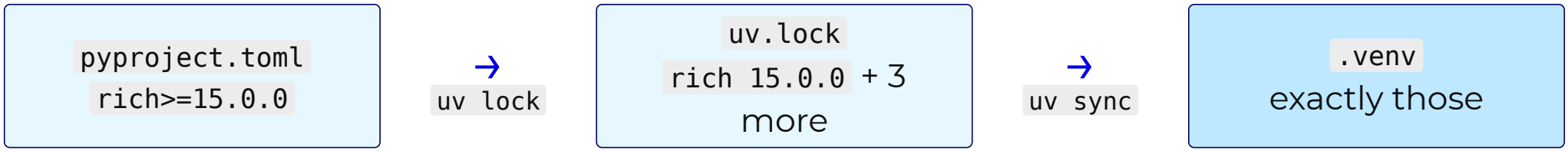
> `>=15.0` today and `>=15.0` next month can install different versions.

The lockfile

`uv.lock` says which versions we **use**: every package, direct **and transitive**, with its exact version and hash. uv writes it.



From constraints to environment



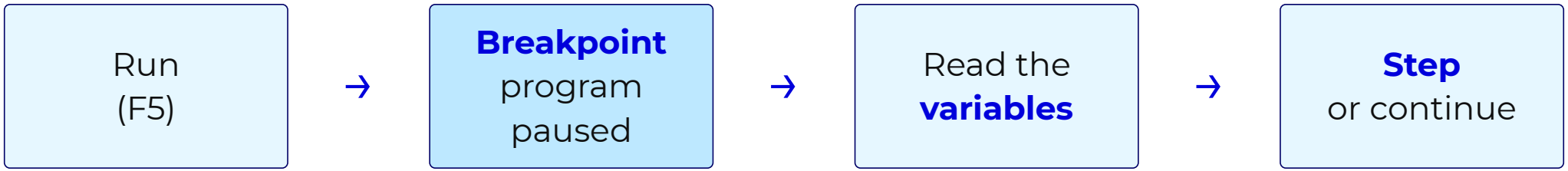
> Commit `pyproject.toml`, `uv.lock` and `.python-version`. Ignore `.venv`.

uv commands for dependencies

`uv add rich`	add a dependency: updates `pyproject.toml`, `uv.lock`, `.venv`
`uv add --dev ruff`	a dependency only for development, in its own group
`uv remove rich`	remove it
`uv sync`	make `.venv` match `uv.lock` exactly
`uv tree` · `uv lock --upgrade`	inspect · upgrade deliberately

Debugger

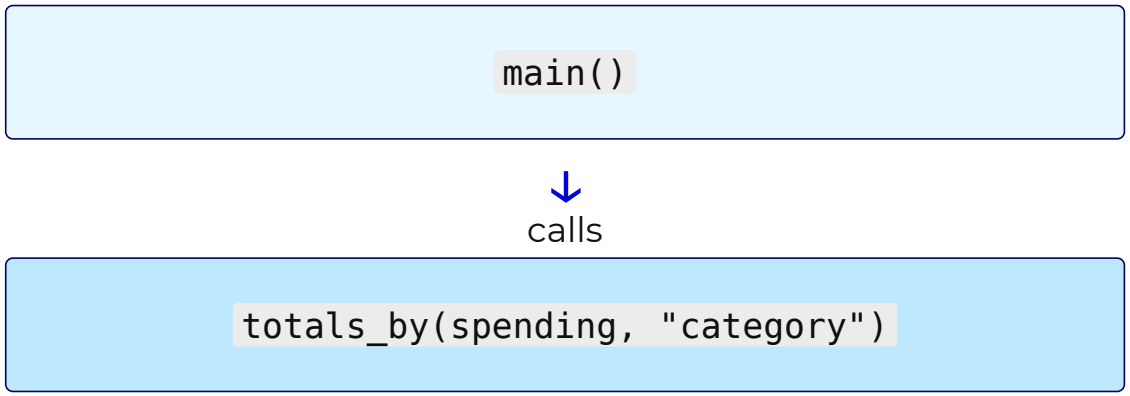
What a debugger does



Instead of adding `print` calls and running again.

> **Conditional breakpoint**: stop only when `t["category"] == "health"`

Stepping through calls



Step Over: run the line, stay here
Step Into: go inside the call
Step Out: finish, back to `main`
Call Stack panel: this chain

launch.json

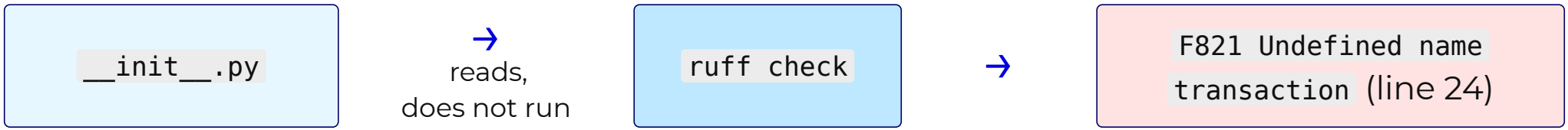
How VS Code runs `ledger spending` under the debugger:

```
{
  "name": "ledger spending",
  "type": "debugpy",
  "request": "launch",
  "program": "${workspaceFolder}/.venv/bin/ledger",
  "args": ["spending"],
  "console": "integratedTerminal"
}
```

> `program`: the command uv created in `.venv/bin`. `args`: what you type after `uv run ledger`.

Linter and formatter

Linter



- Finds problems without running the code
- Some are style, some are bugs
- Python itself only fails when it reaches that line

Formatter

```python
spending = [t for t in transactions if t["amount"] < 0 and t["category"] != "transfer"]
```

↓

`ruff format`

```python
spending = [
    t for t in transactions if t["amount"] < 0 and t["category"] != "transfer"
]
```

Same behaviour, one standard layout: a code review shows only real changes.

> Ruff is both linter and formatter: `uv add --dev ruff`

Lab 4: writing and checking the ledger

Today’s lab

VS Code
write the ledger
`Decimal`

→

`uv add rich`
`uv.lock`
rebuild

→

Debugger
`launch.json`
breakpoints

→

Ruff
`ruff check`
`ruff format`